

High Performance Computing

CS5013

Lab 16 – MPI: Parallel Matrix Multiplication (NxN Floating Point)

R Abinav — ME23B1004

Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 #define N 4
6
7 void print_matrix(float mat[N][N], const char *name) {
8     printf("%s:\n", name);
9     for (int i = 0; i < N; i++) {
10         for (int j = 0; j < N; j++) {
11             printf("%8.2f ", mat[i][j]);
12         }
13         printf("\n");
14     }
15     printf("\n");
16 }
17
18 int main(int argc, char *argv[]) {
19     int rank, size;
20
21     MPI_Init(&argc, &argv);
22     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
23     MPI_Comm_size(MPI_COMM_WORLD, &size);
24
25     int rows_per_proc = N / size;
26
27     float A[N][N], B[N][N], C[N][N];
28     float local_A[N][N], local_C[N][N];
29
30     if (rank == 0) {
31         printf("Initializing %dx%d matrices with float values...\n\n",
32             N, N);
33         for (int i = 0; i < N; i++) {
34             for (int j = 0; j < N; j++) {
35                 A[i][j] = (float)(i * N + j + 1);
36                 B[i][j] = (float)(i == j ? 2.0 : 1.0);
37             }
38         }
39         print_matrix(A, "Matrix A");
40         print_matrix(B, "Matrix B");
41     }
```

```
42 MPI_Scatter(A, rows_per_proc * N, MPI_FLOAT,
43            local_A, rows_per_proc * N, MPI_FLOAT,
44            0, MPI_COMM_WORLD);
45
46 MPI_Bcast(B, N * N, MPI_FLOAT, 0, MPI_COMM_WORLD);
47
48 for (int i = 0; i < rows_per_proc; i++) {
49     for (int j = 0; j < N; j++) {
50         local_C[i][j] = 0.0f;
51         for (int k = 0; k < N; k++) {
52             local_C[i][j] += local_A[i][k] * B[k][j];
53         }
54     }
55 }
56
57 printf("Process %d: computed rows %d to %d\n",
58        rank, rank * rows_per_proc, (rank + 1) * rows_per_proc - 1);
59
60 MPI_Gather(local_C, rows_per_proc * N, MPI_FLOAT,
61            C, rows_per_proc * N, MPI_FLOAT,
62            0, MPI_COMM_WORLD);
63
64 if (rank == 0) {
65     printf("\n");
66     print_matrix(C, "Result Matrix C = A x B");
67 }
68
69 MPI_Finalize();
70 return 0;
71 }
```

Output

```
Initializing 4x4 matrices with float values...
```

```
Matrix A:
```

1.00	2.00	3.00	4.00
5.00	6.00	7.00	8.00
9.00	10.00	11.00	12.00
13.00	14.00	15.00	16.00

```
Matrix B:
```

2.00	1.00	1.00	1.00
1.00	2.00	1.00	1.00
1.00	1.00	2.00	1.00
1.00	1.00	1.00	2.00

```
Process 0: computed rows 0 to 0
```

```
Process 2: computed rows 2 to 2
```

```
Process 1: computed rows 1 to 1
```

```
Process 3: computed rows 3 to 3
```

```
Result Matrix C = A x B:
```

11.00	12.00	13.00	14.00
31.00	32.00	33.00	34.00
51.00	52.00	53.00	54.00
71.00	72.00	73.00	74.00

Figure 1: Program Output – MPI Parallel Matrix Multiplication